

ParallelBayes：时间并行 MCMC 的可核验实现与成本评估

中文研究稿：作者与单位待定

2026 年 10 月

摘要

时间并行马尔可夫链蒙特卡洛 (MCMC) 可以降低指定轨迹的执行延迟，但这种收益并不直接等于可靠推断加速。本文实现研究软件 ParallelBayes，将目标与坐标、转移核、执行器和测量策略分离，保存实际随机数组、独立顺序参考、接受事件及失败记录。软件提供 Stan/BridgeStan 的 CPU 顺序入口，以及明确实现目标的 JAX 和 torch 时间并行路线。在冻结的 M4 Mac CPU 实验中，1920 项任务通过数值核验；两种时间执行器在全部模型—预算组的缓存执行中位数均未获得加速。在另一独立协议的 512 项原生 Windows CPU/CUDA 任务中，RTX 5080 上的 Picard 缓存执行组中位速度比为 1.097–5.230，quasi-DEER 为 0.164–0.505。然而，部分 Picard 收益伴随低接受率或全部拒绝，且大量短链诊断不良，不能据此认定推断优势。解析 SBC 配对对照及跨系统重放进一步表明，输入、轨迹、参数变换与诊断的等价要求必须分别核对。结果提供了指定实现和硬件上的执行收益与推断可靠性边界；同机充分预算下的共同精度比较仍未完成，不声称通用加速或新采样算法。

证据范围。 Mac 主实验使用归档 0.1.0；0.1.1 对应后续 CPU 修订及 SBC。Windows 的 0.2.0 开发版及 512 项任务使用独立协议，见第7节。192 个 Windows 机制 workflow 采用另一个有限协议，每模型两份实际输入。跨系统伴随核验不增加独立统计重复，也不替换历史结果。本文为研究中稿，待完成的实验不计入结果。

1 问题与研究范围

时间并行 MCMC 的吸引力在于缩短单条指定轨迹的计算延迟。轨迹中的相邻状态存在依赖，因此并行方法通常需要推测多个状态，再修正其一致性。已有工作研究了基于 Newton 类固定点求解的 MCMC 轨迹并行，以及基于增量 Picard 映射的 Metropolis 并行 [1, 2]。这些工作提供了算法机制和特定实验中的加速证据；是否能在另一种目标几何、硬件或核验标准下获得完整推断收益，仍须逐项测量。

本研究围绕两个相互关联而不同的问题展开。第一，在核参数和实际随机输入固定时，并行执行能否更快地产生满足规定输出标准的轨迹？第二，允许不同采样核及各自

预热后，后验函数误差与 workflow 成本如何变化？历史 CPU 实验只评价两个固定预算，Windows 首轮则评价两种 MH 核的执行；二者都没有测得精确达标时间。前者控制统计效率，后者同时受初始化、混合和数值计算影响。把二者合成一个速度比会使实现优化与推断效果难以区分。

软件复用现有建模和推断组件。Stan 为连续参数模型提供语义和约束变换 [3]；BridgeStan 暴露已编译模型的密度及导数 [4]；BlackJAX 提供可组合的 JAX 推断组件 [5]；Pyro 提供基于 PyTorch 的概率编程组件 [6]。ParallelBayes 的研究对象是这些组件之上的可核对执行与实验协议。当前 Stan 入口仅支持 CPU 顺序 RWM/MALA，尚未连接 CPU 时间并行执行器；原生 JAX 与 torch 路径均须逐模型实现和核验。torch 后端已在原生 Windows CPU/CUDA 上取得实测证据，其能力不由安装深度学习框架本身推定。软件没有提供新建模语言。本版只考察固定维度、连续参数目标；不声称涵盖全部链内并行、多机后验合并或动态 NUTS 时间并行。

统一调用和跨模型比较已有直接前作。compareMCMCs 支持不同引擎的运行与计时，PPL Bench 提供概率编程基准框架，posteriordb 提供目标、数据和参考样本 [7, 8, 9]。更直接地，ParallelMCMC.jl 已把时间并行 MALA 接入 AbstractMCMC 接口，BayesForge 也提供连接 JAX 推断后端的 R 建模入口 [11, 12]。这些软件文档说明功能重叠，本研究没有复现其性能。本文的证据增量在于将实际随机输入、并行残差、接受事件、无效输出和任务成本共同保存，在同核执行与跨核推断两个层次核对指定实现。它不构成首次库封装、首次 R 硬件加速或通用基准平台的优先权声明。水井案例已接入并核验目标与接口，其正式推断比较和外部模型库的普遍性验证尚未完成。

2 模型、转移核与执行器

2.1 目标与坐标

设原参数为 $\theta = T(q)$ ，计算使用无约束坐标 $q \in \mathbb{R}^d$ ，目标为

$$\ell(q) = \log p(y, T(q)) + \log |\det J_T(q)|. \quad (1)$$

模型记录参数顺序、约束映射、完整数据、实现版本与指纹。两个提供方式可省略不同的参数无关常数，因此以 $\ell(q) - \ell(q_0)$ 、同坐标梯度及变换结果核对目标。正值测试例使用 $\theta = \exp(q)$ 和 lognormal 密度，其 $-\log \theta$ 项与 Jacobian 抵消。这一例同时用于检测遗漏 Jacobian 的实现错误。

R 是建模控制与分析入口。BridgeStan 在 CPU 上调用编译后的 C++ 密度和自动微分，Python 串行循环组织转移；原生 JAX 模型进入 JIT 和批处理；独立 torch 后端使用 NumPy 参考及显式 CPU/CUDA 执行，当前时间执行器仍含 Python 主机控制。Stan 模型指针不跨线程共享，Stan 内部线程关闭。此时高频调用边界的成本属于软件 workflow，不能从跨后端差异直接推断并行算法的贡献。

目标入口与实测设备	顺序 RWM/MALA	Picard RWM / quasi-DEER MALA	NUTS
Stan/BridgeStan, Mac CPU	支持	不支持	不支持
明确 JAX 目标, CPU	支持	支持对应核	BlackJAX 顺序核
明确 torch 目标, Windows CPU	已核验	已核验, 主机控制	独立研究 CLI: Pyro 四链
明确 torch 目标, Windows CUDA	已核验	已核验, 主机控制	未接入/核验

表 1: 当前代码的支持范围与实测边界。CPU Pyro 基线是独立研究 CLI, 不是 `pb_sample()` 的通用 torch NUTS 选项; 四链并发也不是 NUTS 轨迹时间并行。原生 Windows Stan 编译尚未核验。支持数值输出不等于已证实后验精度。

R 函数 `pb_capabilities()` 查询包接口支持的组合; 独立研究 CLI 另列运行契约, 不能从表1推定全部路线都能由一个 R 函数调用。`pb_benchmark()` 只组织配置列表及 R 结果, 正式协议冻结、检查点和完整证据管理依赖 Python CLI 及项目文件。Stan 模型没有默认独立轨迹参考, `audit=TRUE` 明确拒绝。扩展示例通过公开 `Python Model` 提供 Poisson 目标及独立 NumPy 参考, R 经 `reticulate` 调用并转换为 posterior 对象; 并非 `pb_model()` 可直接接收任意 R 闭包。

2.2 固定随机输入下的核

MALA 的步长约定为

$$y = q + h\nabla\ell(q) + \sqrt{2h}z, \quad z \sim N(0, I_d). \quad (2)$$

接受对数比包含正反向提议密度:

$$a(q, y) = \ell(y) - \ell(q) - \frac{\|q - y - h\nabla\ell(y)\|^2 - \|y - q - h\nabla\ell(q)\|^2}{4h}. \quad (3)$$

当 $\log u < a(q, y)$ 时接受, 否则保留 q 。随机游走 Metropolis (RWM) 使用 $y = q + sz$, 接受对数比为 $\ell(y) - \ell(q)$ 。拒绝产生的自环计为一个转移。本文的 h 不等于提议噪声标准差; 这一约定在所有配对执行中固定。

随机输入由锁定 NumPy 版本下的 Philox 生成, 采样与导数探测种子分开。比较固定的是噪声数组和对数均匀随机数组, 而非跨软件的同名整数 seed。正式原始输出保存这些数组及其校验和。独立顺序 oracle 使用 NumPy 密度、解析梯度和显式循环, 不调用待核验的 JAX 或 torch 转移函数。跨系统重建派生浮点数组不保证逐位一致; 重复执行应传递已封存的实际数组, 再分别核对文件与数组哈希。

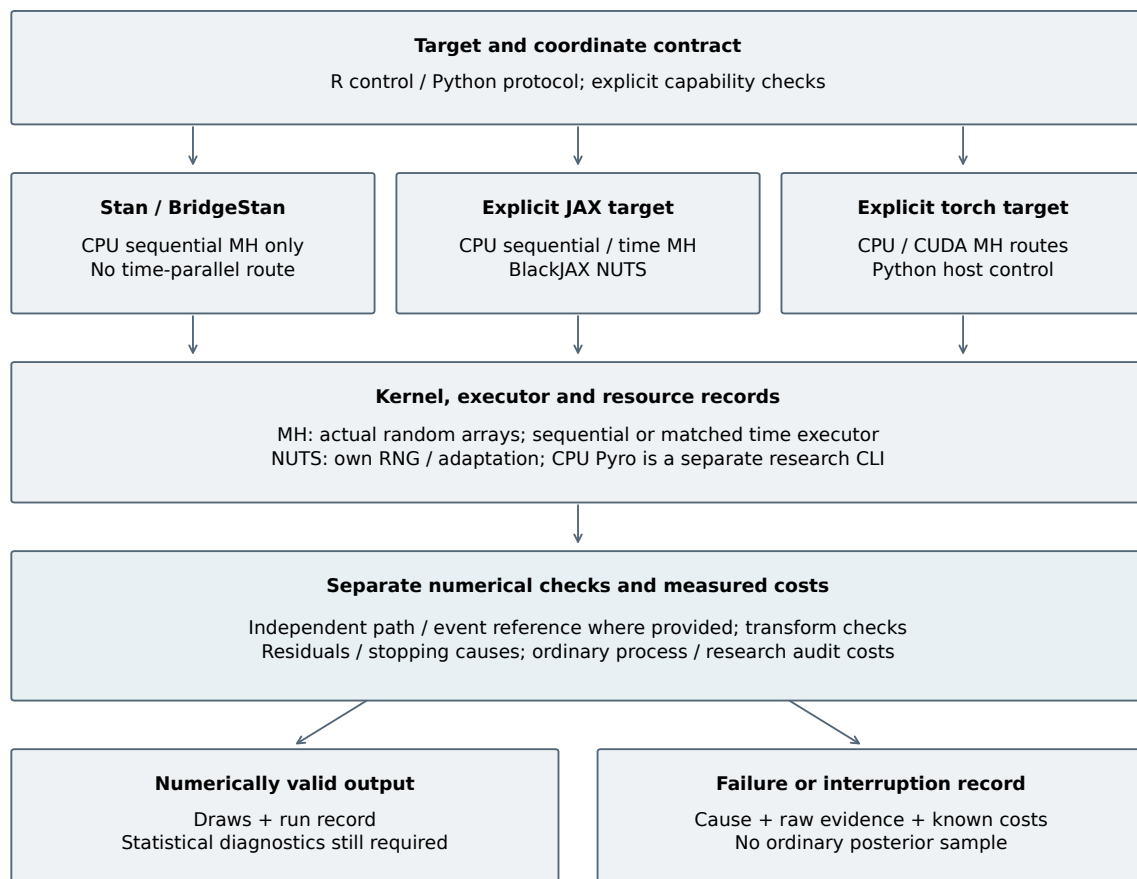


图 1: 当前软件路线与证据流。Stan 只连接 CPU 顺序核; JAX 和 torch 仅接受明确实现的目标。独立顺序参考须另外提供, Stan 入口没有默认参考。NUTS 使用自身随机流及预热, 独立 Pyro CLI 另作记录。数值有效输出继续接受统计诊断, 失败不能转为正常样本。图示不含性能预测。

2.3 两类时间执行

quasi-DEER 执行器使用非重叠窗口。每轮以 Rademacher 向量和 Jacobian-向量积估计转移映射的对角导数，截断导数后通过仿射结合扫描更新窗口路径。接受事件在前向计算中仍是硬分支，导数采用 sigmoid 直通构造。与上游实现相比，本版改变了窗口调度和停止判据，且不使用 NaN 归零或状态裁剪；因此称为基于 quasi-DEER 的适配实现，不将其性能等同原论文。

设 F_t 为固定实际随机输入下的硬转移映射，数值停止要求

$$r = \max_{t,j} \frac{|q_{t,j} - F_t(q_{t-1})_j|}{\text{atol} + \text{rtol}|F_t(q_{t-1})_j|} \leq 1. \quad (4)$$

这里的残差控制局部递推一致性，既不是整条路径误差界，也不是统计无偏性证明。正式实验同时执行独立路径和接受事件核验，容差设为 $\text{atol} = \text{rtol} = 10^{-10}$ 。独立审计另要求接受事件零失配，且每链无约束路径最大绝对差不超过 $100[\text{atol} + \text{rtol} \max(1, \max |q|)]$ ；100 倍系数是预先固定的数值检查阈值，不是传播界。容差求解与独立核验成本均保留。

Online Picard 根据增量映射独立实现。固定 RWM 噪声后，接受增量只能为 sz_t 或零；并行计算旧猜测下的增量，经前缀和产生候选路径，再在新前驱状态下复核接受事件。最长一致增量前缀用于推进滑动窗口。数学上的增量一致性与浮点逐位相等不同，本文只报告数值路径差和事件一致性。

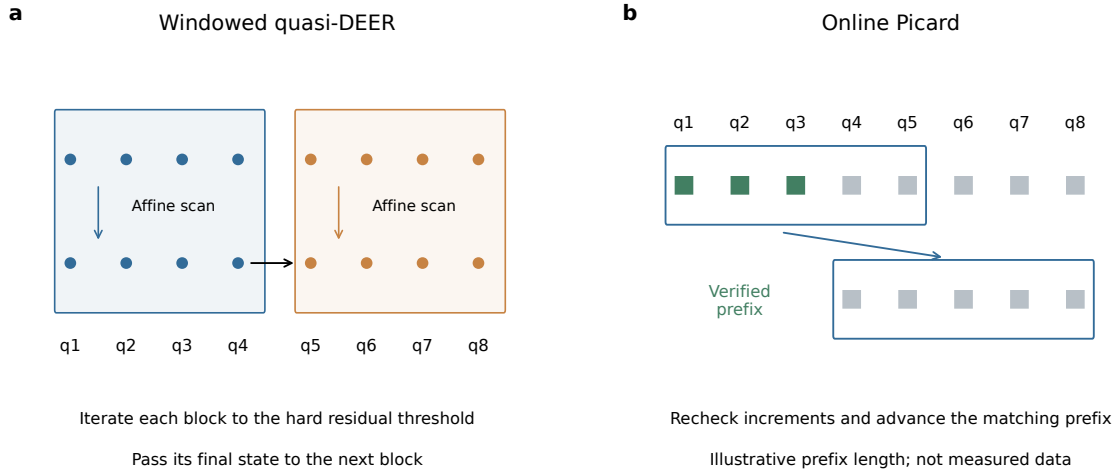


图 2: 两种执行器的调度示意。左侧按非重叠窗口迭代，每轮通过仿射扫描更新该窗口路径，停止后向下一窗口传递终点。右侧先并行计算增量，再根据新前驱复核接受事件，并推进最长一致前缀。绿色表示示意中确认的前缀。图中的窗口和前缀长度仅用于解释，不是实测轨迹，也不暗示固定迭代次数。

2.4 失败与结果对象

达到迭代上限、出现非有限状态或核验失配的轨迹不进入正常抽样对象。结果记录原始失败轨迹、残差、接受事件、求解次数、导数裁剪、确认长度和停止原因。MH 原型将非有限密度、梯度或提议视为失败；首版未一般地区分合法零密度提议与数值溢出。NUTS 保留所用 BlackJAX 或 Pyro 基线的发散诊断，并在返回样本有限时保留其输出。两类数值事件分别报告，不能用统一的“completed”标签暗示收敛。

用户可以显式选择执行器停止失败后的顺序回退；独立审计失配或变换错误不触发该选项。回退复用原随机数组重新计算整个指定轨迹，保留并行阶段的失败及成本；不重抽随机数，也不选择较快完成的链。本文正式比较采用失败保留策略，不通过回退隐藏并行求解失败。约束输出仍须检查形状、有限性及可表示的支持集；审查中发现的边界处理修复以独立软件版本交付，冻结实验的源码归档保留。R 返回 `iteration × chain × variable` 排列的 `posterior::draws_array`，与独立运行记录配套。

3 历史 CPU 验证与开发证据

软件测试覆盖密度、梯度、Jacobian、约束输出、固定噪声路径、接受事件、拒绝自环、部分窗口、迭代耗尽、非有限目标、内存守卫和共同随机输入回退。错误注入包括遗漏 Jacobian、改变尺度以及改变实际目标造成接受事件失配。协议与文件校验测试还检查重复任务跳过和输出损坏检测。有限测试不能替代一般正确性证明，但提供了可重放的反例检测机制。

冻结 0.1.0 通过 43 项 Python 测试和 R 包检查。审查修订版 0.1.1 进一步通过 58 项 Python 检查、R 包检查及 R 到 BridgeStan 的端到端核验；后者的固定输入样本差为零。CPU 主实验仍对应归档的 0.1.0，新版边界修复不覆盖历史输出。高斯、logistic 与正值模型的 Stan/原生实现交叉核验通过；两个简单高斯目标上，上游固定提交的求解器与独立 MALA 参考的路径差小于 10^{-8} 。这项复现仅检验上游求解器与指定映射的组合，不声称复现原文全部性能实验。

开发探测使用 G1、G2、L1、L2，固定转移数为 256 或 2048，窗口为 16、64、256，每组采用三个实际随机输入数组。192 项任务均完成，最大路径差为 2.17×10^{-9} 。Mac CPU 上的并行执行通常增加了时间；尤其是大数据 logistic 的多次梯度与导数求值成本明显。该探测属于开发资料，最早若干任务与初次 Stan 编译重叠，因此不用于正式速度推断。

小型生成式校准检查从 $\theta \sim N(0, 1)$ 及八个 $y_i \sim N(\theta, 1)$ 生成 12 个独立数据集，每个数据集运行五种工作流。后验具有解析均值 $\sum_i y_i / 9$ 和标准差 $1/3$ 。60 次拟合均完成，MALA 的顺序/并行秩直方图相同，RWM 的顺序/并行结果也相同。该重复数下，观察到的区间覆盖不能确证校准；秩抽样的相关性和有限功效须继续检查。正式 SBC 使用另一组 64 个生成数据集和独立种子，避免把已观察的开发样本作为未见验证资料。

在正式 SBC 拟合开始前，另冻结依赖数据的对数似然统计量 $f_y(\theta) = -\frac{1}{2} \sum_i (y_i -$

$\theta)^2$ ，并从同一批保存样本计算其秩和解析后验期望误差。秩每 32 步取一个样本，共 64 个；与真参数处统计量相等的样本单独计数，以独立 Philox 流在并列范围内随机化秩。此伴随分析不改变拟合协议，也不据有限、仍可能相关的秩宣布一般校准通过。

4 历史 Mac CPU 的冻结实验设计

4.1 目标族与基线

目标	定义	比较作用
G1	8 维标准高斯	简单几何与执行开销
G2	64 维旋转高斯，条件数 100	耦合与尺度差异
L1/L2	8 参数 logistic，样本量 256/4096	似然计算量
H1/H2	9 维漏斗，中心化/非中心化	相同分布的坐标与预热效应
A1	64 维潜在 AR(1) 高斯后验	高维相关结构
M1	8 维对称双峰高斯混合	跨模式探索压力

表 2: Mac CPU 主协议的目标库。H1/H2 属于同一模型族；L1/L2 是同类回归的计算量变化。数据由记录的随机规则生成。本研究不把这些变体当作独立应用领域。

五种工作流为顺序 MALA、窗口 quasi-DEER MALA、顺序 RWM、Online Picard RWM 和 BlackJAX NUTS。各工作流均采用四链；MH 使用预先固定步长，前 512 步作为丢弃段，随后保留 256 或 2048 步。NUTS 单独预热 512 步后保留相同数量，最大树深为 8。所有初始化固定为零，步长表、数据、随机种子及运行顺序写入机器可读协议。比较的对象是这些指定工作流，不能外推为经过全局最优调参的算法排名。

正式协议在独立探测后冻结，每组 24 次独立重复，共 1920 项任务。主实验不依据有利结果追加重复或选择模型。原研究计划提出的误差 MCSE 相对阈值 0.1 并未被承诺对每个压力目标满足；本版冻结有限重复数并完整报告其不确定性，区间跨阈值时保留为未判定。没有训练自动配置选择器，也没有将已见模型声明为选择器留出集。

4.2 误差与参考

对预先规定的函数集合 $\mathcal{F}$ ，以独立重复评价

$$\hat{E}_a(N) = \max_{f \in \mathcal{F}} \frac{1}{R} \sum_{r=1}^R \left(\hat{\mu}_{a,f}^{(r)}(N) - \mu_f \right)^2 / s_f^2. \quad (5)$$

高斯目标使用标准化首坐标的均值、二阶矩及超过 1 的概率。漏斗目标使用标准化尺度变量、其双曲正切、局部变量的符号事件及条件标准化变量的余弦；这避免依赖漏斗局部变量的巨大高阶矩。双峰目标同时检查模式占比和连续函数。Logistic 检查前两个参数、截距的 logistic 变换及符号事件。全部函数与尺度在协议中明确，尺度统一取 1。

解析目标使用真实期望。L1/L2 的参考分别来自四次独立 NUTS 拟合，每次四链、预热 1024 步、保留 16384 步；参考种子与正式实验分离。两目标参考均未记录发散，L1 的所检查函数及 L2 的连续函数 $\text{split-}\hat{R}$ 接近 1，非退化函数参考 MCSE 最大约为 7.6×10^{-4} 。原协议计算所选函数的经典 $\text{split-}\hat{R}$ ；审查后从相同保存样本补充秩正态化/折叠 $\text{split-}\hat{R}$ 、bulk ESS 和 tail ESS，后文单列，不替换原分析。L2 符号事件在全部参考样本中为零，其 Rhat 不可判定、参考误差未确定；不把零经验方差当作已证实的精度。参考 MCSE 取拟合间估计与批均值估计的较大值；该规则并不排除共同偏差。对这两个目标的结果称为参考平方差，不把它解释为无偏真实 MSE，也不无条件减去参考方差。

不确定性采用独立重复层面的 4000 次重采样，报告逐点 95% 区间；每次重采样先在重复间求平方误差均值，再在预声明函数间取最大值。对可用共同参考另做正态、对角协方差扰动的敏感性传播，同一次扰动在所有重复中共用。它没有估计函数之间的完整参考协方差，不等于联合误差的精确传播；基本重复 bootstrap 区间另行保留。区间不是跨模型、方法和预算的同时置信保证。失败率、成功输出上的条件误差及全部成本分别呈现，有失败时不宣称无条件精度达标。

4.3 计时与资源

本地实验在 M4 Mac mini、16 GB 内存和 JAX CPU 后端上执行，使用 float64、JAX 0.6.2、BlackJAX 1.2.5、NumPy 2.2.6 和 BridgeStan 2.7.0。外层同一时刻运行一个实验任务。目标平台是整台指定机器，未将逻辑 CPU 个数直接解释为独占物理核预算。正式计算期间同机还进行了少量软件核验与文档生成，未实现全程独占空闲系统；保留全部原任务，不依据这种背景活动剔除计时值。

显式同步设备后结束执行计时。记录 JIT 编译、输入、NUTS 预热、执行、传输、独立核验和回退；逐任务墙钟还包含建模和写盘。本文以逐任务墙钟扣除额外计时重放成本、加上后验函数摘要与诊断计算时间，定义完整工作流程时间；建模、原始数组写盘和校验和计算均在该口径内，另保留纯采样接口计时。完成初次执行后对同一编译程序进行两次缓存内重放，作为执行机制计时探测，额外成本单列；它们不是新的统计重复，也不计作增加后验样本。进程导入和一次性软件安装不是单次任务计时的一部分，因此本文不将该口径称为全新进程冷启动。

MH 的步长与窗口作为指定工作流的输入；开发探测属于离线研究费用，单独保留，不在每个正式拟合中重复摊入。NUTS 的逐次适配则包含在预热成本中。因此完整工作流程比较回答的是给定配置的运行成本，尚未覆盖未知新模型上选择 MH 参数所需的成本，不应称为自动调参方案之间的完整费用比较。

CPU 内存为采样 API 期间每 20 ms 记录的进程绝对 RSS 峰值，包含先前驻留分配，未覆盖建模、写盘和子进程；该 CPU 协议不评价 GPU 内存；Windows 另有设备分配器记录。数组空间有事前保守预算检查，数值迭代上限保留。运行没有人为总时长截止。冻结源代码和协议校验和、随机化任务顺序、逐任务原始数组与状态支持中断后

恢复；成功与失败结果均保留。

5 历史 CPU 结果与范围

本机按冻结协议登记了 1920 项正式任务，全部结束，其中 0 项未返回满足输出标准的正常样本。失败计数与原始输出均保留；该总任务数包含配对执行，并非 1920 个独立模型。每个模型、预算及工作流的独立重复数为 24。

正常输出中，独立核验记录的最大数值路径差为 2.36×10^{-8} ，接受事件失配总数为 0。这一陈述以数值核验及当前目标库为边界，不构成一般轨迹或平稳分布证明。

quasi-DEER MALA 的模型—预算组中位缓存执行速度比范围为 0.0192 至 0.174，研究审计工作流速度比范围为 0.0715 至 0.739。速度比定义为同核顺序时间除以并行时间，小于 1 表示并行执行更慢；这里只统计成对有效输出，失败另行报告。

Online Picard RWM 的模型—预算组中位缓存执行速度比范围为 0.0503 至 0.809，研究审计工作流速度比范围为 0.226 至 0.814。速度比定义为同核顺序时间除以并行时间，小于 1 表示并行执行更慢；这里只统计成对有效输出，失败另行报告。

表中列出较大预算下的函数最大平方误差。L1/L2 采用有限参考，因此相应数值是参考平方差。完整逐函数误差、逐点区间、参考敏感性区间、失败比例区间及两预算结果随机器可读材料交付。不能把表中的最小数值自动解释为确定排名。

目标	顺序 MALA	quasi-DEER	顺序 RWM	Picard	NUTS
G1	0.00199 (0)	0.00199 (0)	0.00542 (0)	0.00542 (0)	0.000531 (0)
G2	0.125 (0)	0.125 (0)	0.164 (0)	0.164 (0)	0.000322 (0)
L1	0.000255 (0)	0.000255 (0)	0.000673 (0)	0.000673 (0)	1.46×10^{-5} (0)
L2	$6.22 \times 10^{-6} \uparrow$ (0)	$6.22 \times 10^{-6} \uparrow$ (0)	$1.91 \times 10^{-5} \uparrow$ (0)	$1.91 \times 10^{-5} \uparrow$ (0)	$2.28 \times 10^{-7} \uparrow$ (0)
H1	0.138 (0)	0.138 (0)	0.131 (0)	0.131 (0)	0.112 (0)
H2	0.0215 (0)	0.0215 (0)	0.0376 (0)	0.0376 (0)	6.11×10^{-5} (0)
A1	0.00803 (0)	0.00803 (0)	0.856 (0)	0.856 (0)	0.000665 (0)
M1	0.252 (0)	0.252 (0)	0.359 (0)	0.359 (0)	0.296 (0)

表 3: 每链保留 2048 步时的最大函数平方误差/参考平方差。括号内为该组 24 次重复中的输出失败数；有失败时，误差仅描述成功输出。误差不包含失败的假定损失，不代表无条件达标。 $\uparrow$ 表示参考精度未确定，该行仅展示参考平方差，不能判断全函数精度达标。

按前述工作流口径汇总，正式尝试合计耗时 5080.9 秒，其中失败任务耗时 0.0 秒。另有缓存计时重放开销 5711.5 秒，完整保留但不重复计入单次工作流成本。逐组成本及全部失败分母另存分析结果。

NUTS 在 46 个已返回有限样本的任务中记录到发散，总计 2358 个保留转移的发散事件。发散与数值输出失败分别呈现，不能因为数组有限而删除这一诊断。多峰和漏

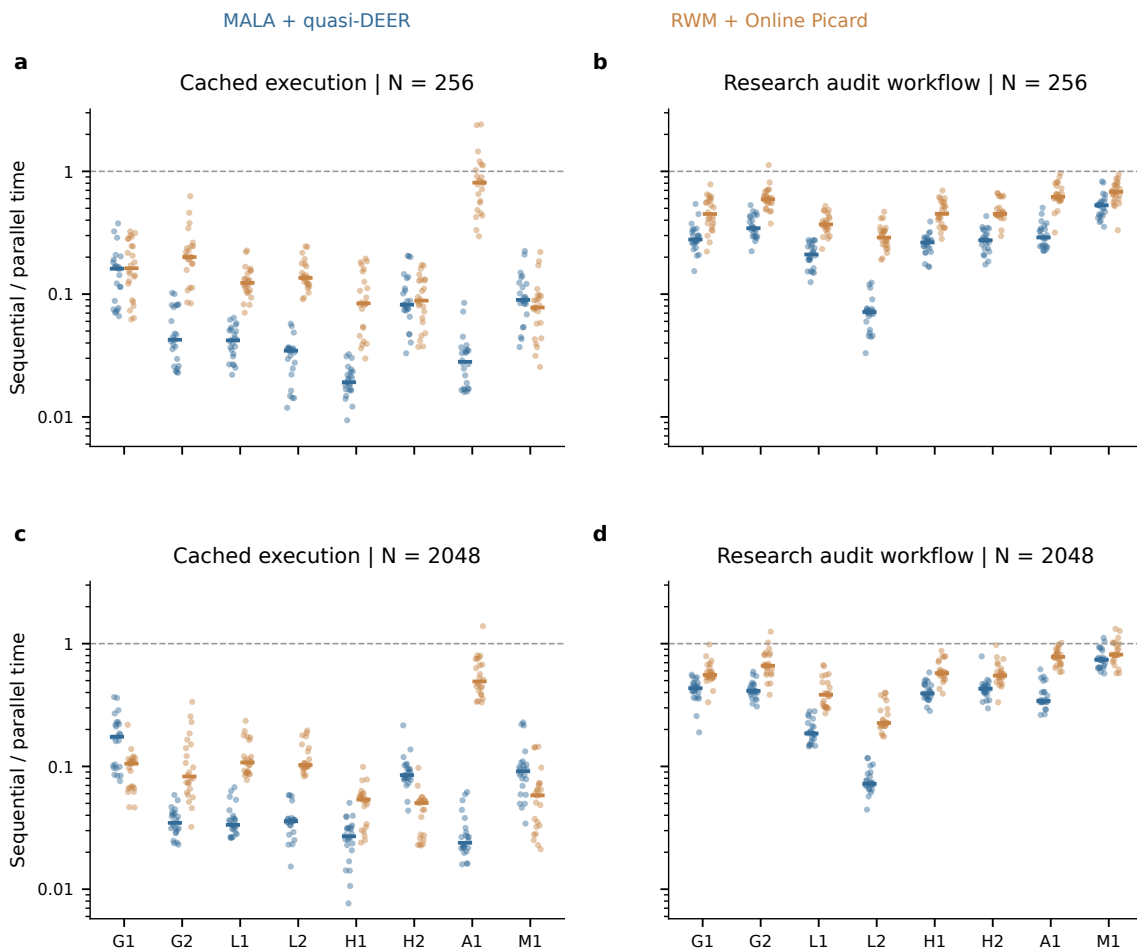


图 3: 同核成对成本。上、下行分别为每链保留 256 和 2048 步; 左列为两次缓存重放时间的中位数之比, 右列为含编译、预热/丢弃段和独立核验的工作流时间比。每个点为一个独立重复中的配对比, 短线为组中位数; 同一轨迹的计时重放不当作统计重复。虚线表示速度比 1。任何缺失配对均在原始任务及失败表中保留。

目标上的误差须结合模式占比及尺度函数理解。

NUTS 另有 6861 个保留转移达到 255 个积分步的设定上限；积分步数及逐次适配参数随原始记录保存。

采样接口期间记录的逐任务进程 RSS 峰值范围为 244.3 至 549.0 MiB；它包含进程已驻留内存，只覆盖接口调用阶段，不能解释为每种算法的独占增量内存或整项任务的严格峰值。

quasi-DEER 的模型—预算组中，单个输出转移对应的前向转移映射求值次数中位数范围为 14.58 至 66.15，顺序 MH 为 1。

Online Picard 的模型—预算组中，单个输出转移对应的前向转移映射求值次数中位数范围为 2.00 至 21.38，顺序 MH 为 1。

这些程序层面的映射调用计数包含丢弃段和拒绝自环，尚不包含独立审计；它们不是浮点运算量或密度/梯度调用次数。quasi-DEER 的 JVP 另存，因此不能仅据调用数精确分解运行瓶颈。

mala/sequential 在较长预算、 $\epsilon = 0.1$ 下的逐点判断为：区间低于阈值 2 个目标，区间高于阈值 4 个，跨阈值 1 个，参考或输出条件不足 1 个。

mala/quasi_deer 在较长预算、 $\epsilon = 0.1$ 下的逐点判断为：区间低于阈值 2 个目标，区间高于阈值 4 个，跨阈值 1 个，参考或输出条件不足 1 个。

rwm/sequential 在较长预算、 $\epsilon = 0.1$ 下的逐点判断为：区间低于阈值 2 个目标，区间高于阈值 5 个，跨阈值 0 个，参考或输出条件不足 1 个。

rwm/online_picard 在较长预算、 $\epsilon = 0.1$ 下的逐点判断为：区间低于阈值 2 个目标，区间高于阈值 5 个，跨阈值 0 个，参考或输出条件不足 1 个。

nuts/sequential 在较长预算、 $\epsilon = 0.1$ 下的逐点判断为：区间低于阈值 5 个目标，区间高于阈值 2 个，跨阈值 0 个，参考或输出条件不足 1 个。

这些判断仅对应两个已测量预算中的较长预算，不构成同时置信结论，也不提供单次运行可观测的停止规则或精确的达标时间。

正式生成式检查完成 64 个新数据集、五种工作流，共 320 次拟合，其中 0 次输出失败。90% 后验区间的覆盖估计及其二项区间按数据集作为独立单位报告；保留所有秩与原始样本。抽稀秩仍可能相关，这一检查不提供一般 SBC 一致性证明。

mala/sequential: 覆盖率 0.844, 95% 二项区间 [0.731, 0.922]。

mala/quasi_deer: 覆盖率 0.844, 95% 二项区间 [0.731, 0.922]。

rwm/sequential: 覆盖率 0.828, 95% 二项区间 [0.713, 0.911]。

rwm/online_picard: 覆盖率 0.828, 95% 二项区间 [0.713, 0.911]。

nuts/sequential: 覆盖率 0.828, 95% 二项区间 [0.713, 0.911]。

正式生成式检查另记录到 0 个保留转移的发散事件，涉及 0 次拟合；该诊断与输出失败分开保存。

本次覆盖率点估计均低于名义 90%，而相应二项区间均包含 90%。这一有限重复结果既不能证明完全校准，也不应被改写为已确证系统性覆盖不足。

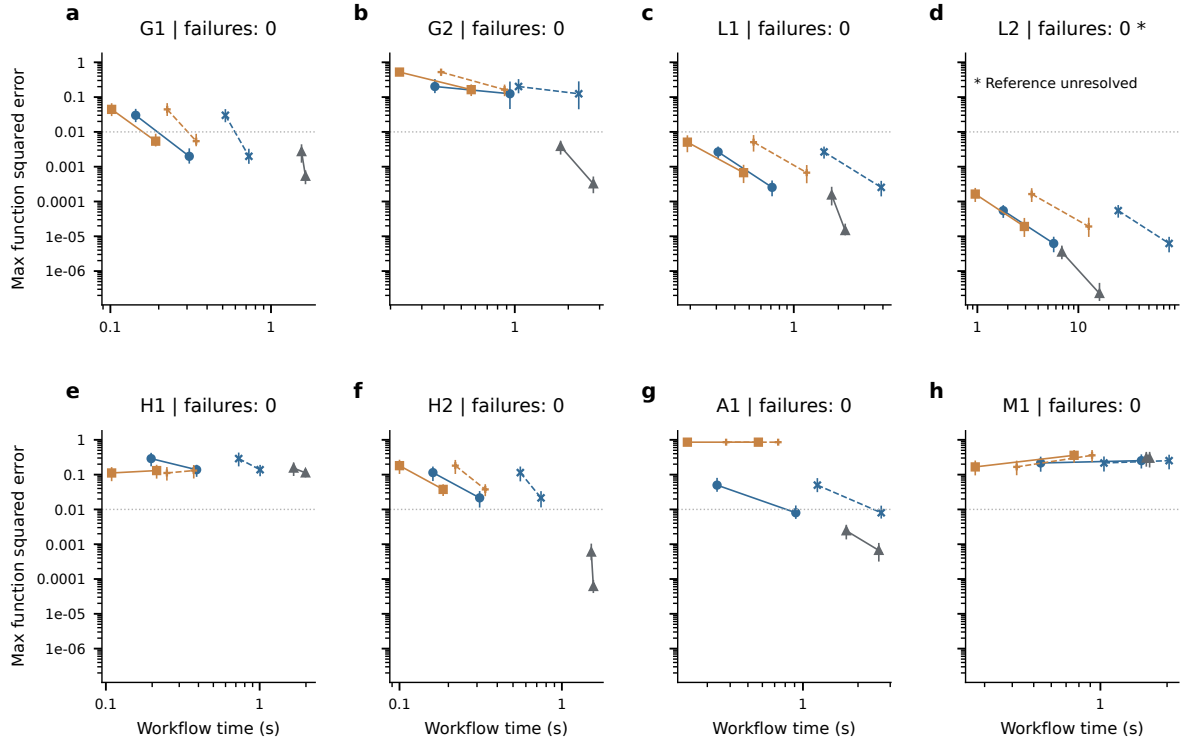


图 4: 固定采样计划的误差—成本关系。横轴为成功输出条件下的 workflow 时间中位数, 全部尝试和失败成本另表保存。每条线连接同一指定工作流的两点预算点, 不表示中间预算已被测量。蓝色为 MALA、橙色为 RWM、灰色为 NUTS; 实线/圆或方形为顺序 MH, 虚线/叉号为相应时间执行器, 三角形为 NUTS。误差取预声明函数中最大的重复平均平方误差; L1/L2 为有限参考平方差。线段区间为逐点 95% 重复重采样区间, 有限参考的共同误差敏感性另行传播; 传播采用正态对角协方差工作假设, 非同时保证。星号表示参考精度未确定, 该面板仅供探索性平方差比较。面板标题列出该模型全部正式任务中的输出失败数, 失败组的误差为条件结果。点虚线标出 0.01 作为预声明误差阈值。

对数似然测试量也完成 320 次拟合的伴随分析，共记录 0 个秩并列。逐方法秩直方图、抽稀后相关性及解析期望误差随原始数据交付；这些描述性诊断不构成统一性检验或正确性证书。

L2 的参考符号事件在 262144 个相关样本中均为零，Rhat 对该函数不可判定，不能把零经验方差当作事件概率已被精确确定。因此保留原函数集合，并将 L2 全函数精度判定标记为未确定。没有通过删除该函数得到达标结论。

上述结果来自历史 Mac CPU 协议，不包含后续 Windows 实验；原生 Windows 实现及其独立协议的实测结果在后文另行报告。

6 审查后的伴随分析与 CPU 机制实验

6.1 版本与扩展验证

CPU 主实验保留 0.1.0 源代码和原协议；0.1.1 增加约束输出检查、未窗口有效步掩码、错误及失败记录，并修复 R 单元数组。变换检查和掩码可能影响计时，不能用旧性能代表新版。审查后对 8 目标、5 工作流各取原较短预算的第一个重复，旧/新源码分别执行；MH 复用保存的实际随机数组，NUTS 复用锁定随机流。40 组均返回有效输出，最大路径差为 1.99×10^{-13} ，接受事件零失配。这是有限数值对照，不是性能等价或边界正确性证明。

新增 Poisson 回归示例无需改动内核，公开模型对象分别实现 JAX 密度与独立 NumPy 密度/解析梯度，先验证坐标和导数，再比较固定输入的四组 MH 工作流。密度与梯度最大差约 2.84×10^{-14} ；MALA 和 RWM 配对最大路径差分别为 7.90×10^{-11} 和 3.61×10^{-16} ，接受事件无失配。示例旨在说明扩展和核验，不作为新性能或后验收敛证据。

6.2 同数据集的解析 SBC 对照

本节为审查后伴随分析，未追溯为原协议预声明检验。64 组真参数和数据已保存，后验为 $N(\sum_j y_j/9, 1/9)$ ，故直接计算解析 90% 区间，无需新增拟合。解析区间覆盖 54/64，点估计 0.84375，95% 二项区间为 [0.731, 0.922]。五种工作流共享同 64 组数据，同核顺序/并行还共享轨迹，因此不能当作五次独立校准证据。

MALA 的分歧数据集编号为 23、32、36、56，RWM 为 26、32、56，NUTS 为 11、26、36（从 0 编号）。例如 MALA 有 2 例解析覆盖而 MCMC 未覆盖、另 2 例相反，抵消后总覆盖数相同。该批解析覆盖本身偏低，与有限生成样本波动的解释相容；端点及后验矩误差仍显示有限链估计差异，不能据此证明无额外采样偏差。

工作流	覆盖数	分歧数	下端点 RMSE	上端点 RMSE	均值 RMSE	标准差 RMSE
顺序 MALA	54	4	0.0179	0.0195	0.0105	0.0063
quasi-DEER	54	4	0.0179	0.0195	0.0105	0.0063
顺序 RWM	53	3	0.0318	0.0267	0.0147	0.0107
Picard	53	3	0.0318	0.0267	0.0147	0.0107
NUTS	53	3	0.0222	0.0190	0.0115	0.0078

表 4: 同 64 数据集的配对解析对照。RMSE 按数据集计算；分歧指解析与 MCMC 区间对同一真参数的覆盖判断不同，不能只据总体覆盖率相等判定区间相等。

6.3 现代诊断与有限预算误差的对应

从全部 1920 项保存样本及 8 次参考拟合提取每个参数和预声明函数，以 posterior 1.7.0 计算秩正态化/折叠 split- $\hat{R}$ 、bulk ESS 与 tail ESS[10]。每个拟合分别计算，结果不替换原经典函数诊断；常量链等不可判定值保留为空。表中汇总 NUTS，全 80 组的逐拟合、逐变量诊断随复现材料交付。

目标/ N	发散%	上限%	最大 $\hat{R}$	最小 bulk ESS	最小 tail ESS	函数误差	判据
A1/256	0.00	0.00	1.018	496.4	469.7	0.00243	低于阈值
A1/2048	0.00	0.00	1.002	5576.0	5186.4	0.000665	低于阈值
G1/256	0.00	0.00	1.018	1198.4	461.8	0.00271	低于阈值
G1/2048	0.00	0.00	1.003	11148.1	5316.6	0.000531	低于阈值
G2/256	0.00	0.00	1.015	636.5	448.0	0.00383	低于阈值
G2/2048	0.00	0.00	1.002	5848.5	5429.4	0.000322	低于阈值
H1/256	0.71	2.67	1.745	6.3	8.0	0.154	高于阈值
H1/2048	1.11	3.16	1.273	11.4	5.4	0.112	高于阈值
H2/256	0.00	0.00	1.018	815.5	545.5	0.000595	低于阈值
H2/2048	0.00	0.00	1.002	7125.8	5438.9	6.11e-05	低于阈值
L1/256	0.00	0.00	1.017	358.2	393.2	0.000154	低于阈值
L1/2048	0.00	0.00	1.003	3459.7	4381.6	1.46e-05	低于阈值
L2/256	0.00	0.00	1.019	507.4	532.0	3.52e-06	不可判定
L2/2048	0.00	0.00	1.002	4243.8	5109.4	2.28e-07	不可判定
M1/256	0.00	0.00	1.738	6.2	30.6	0.301	高于阈值
M1/2048	0.00	0.00	1.734	6.1	26.8	0.296	高于阈值

表 5: NUTS 按模型和保留预算分组的伴随诊断。发散和上限命中以 24 次拟合、各 4 链的全部保留转移为分母； $\hat{R}$ 为参数与拟合中的最大值，ESS 为最小值。函数误差为原预声明最大平方误差/参考平方差，判据对应原逐点 95% 区间与 0.01 阈值，不由诊断阈值重定义。

2358 次发散与 6861 次积分上限命中均来自中心化漏斗 H1：短/长预算分别为 174/2184 次发散及 657/6204 次上限命中。双峰 M1 无发散，却在两个预算下都有约 1.73 的最大参数 $\hat{R}$ ，所选函数误差亦高于阈值。五个目标的长预算函数误差区间低于阈值，与 H1/M1 的探索问题分别成立；L2 参数诊断良好不消除参考符号事件恒零的不确定性。特定函数误差小、无发散和后验全面探索是不同陈述。

6.4 三种成本口径

缓存执行衡量已编译内核；正常推断应包含必要建模、编译、预热、转换及诊断；研究审计另加入独立顺序参考和完整数组、校验和归档。历史时钟未单独测量正常输出与所有建模步骤，故只能以 $T_{\text{API}} - T_{\text{independent}} + T_{\text{diagnostic}}$ 给出内存内正常使用的记账估计，不能冒称重新实测的完整正常工作流。该估计不含未分离的建模、R 转换和持久化费用。

执行器	缓存执行	正常内存口径估计	研究审计工作流
quasi-DEER	0.019–0.174	0.038–0.175	0.072–0.739
Picard	0.050–0.809	0.110–0.393	0.226–0.814

表 6: 历史主实验中，各模型–预算组的配对速度比中位数范围。三列均为顺序时间/并行时间；中列是不同包含项的记账敏感性分析，不与另外两列混称三种完整实测工作流。

共享审计成本使速度比向 1 靠近，可能掩盖加速或减速；当前缓存与去独立核验的估计口径也均未显示组中位加速，因此负结果不能仅归因于审计。新增机制实验另直接计量 `audit=False` 的建模、采样与基本均值/方差/链间诊断，并单列普通样本输出耗时；它是最小 Python 正常工作流，不包含 R 转换及现代 posterior 诊断，也不是全新进程冷启动。

6.5 小型 CPU 机制实验

审查后另冻结 `cpu-mechanism-v1`，使用 0.1.1，目标 G1/G2/L1、链数 1/4、窗口 16/64、每链 256 步、两组成对 MH 执行方式和每格 3 个新实际随机数组，共 72 项任务。沿用主协议的指定核参数，冻结随机顺序；没有按速度追加或删除任务。每次至少 7 次缓存计时重放、总观测窗口至少 0.2 秒，作为技术重复而非独立统计重复。全部 72 项完成；分段重放与生产程序首链的接受事件无失配。

生产执行另外记录进程 CPU 秒/墙钟秒和 10 毫秒间隔系统利用率。组中位进程占用约为 1.00–2.99 个 CPU 核当量，测量包含运行返回和监测开销，不是物理核心绑定，也不能将 `vmap` 当作指定核心数的证明。固定前驱状态的微基准比较串行映射与向量化映射在同一批状态、噪声上的吞吐；它没有 MCMC 的递推依赖，不能将吞吐比直接当作轨迹加速比。

quasi-DEER 的组中位缓存速度比为 0.0145–0.23，固定状态批量吞吐比为 0.381–1.91；每转移前向映射数为 12–45.5，另有 5.5–22.2 次 JVP。

Picard 的组中位缓存速度比为 0.0477–0.732，固定状态批量吞吐比为 0.244–2.46；每转移前向映射数为 2–24.5。

这些量显示局部批量求值收益与完整轨迹重复工作之间的差距，而非证明单一瓶

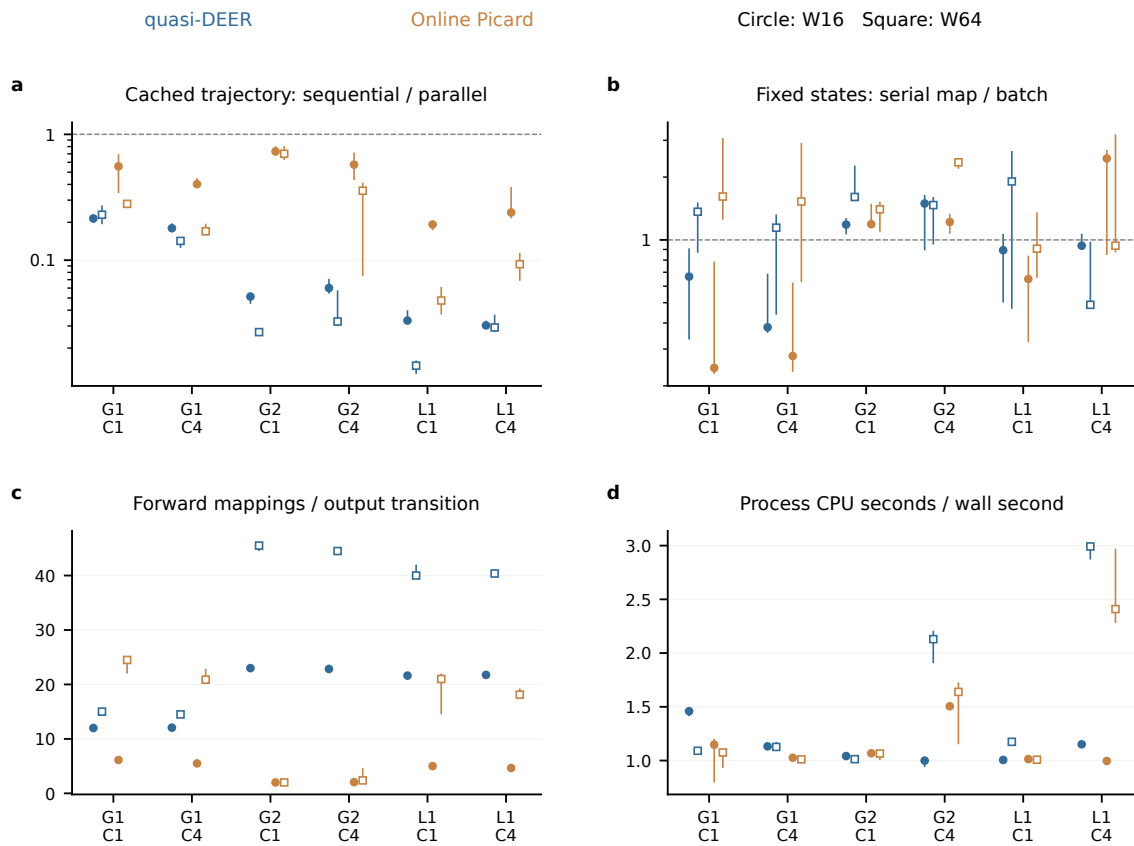


图 5: 新冻结的 72 项机制任务。每格 3 个实际随机数组，符号为组中位数，细线为最小至最大值；技术计时重复不扩大样本量。蓝色为 quasi-DEER，橙色为 Picard；圆点为窗口 16，方形为 64；横轴 C 为链数。a: 缓存轨迹速度比，1 以上才为加速。b: 固定状态批量吞吐比，1 以上表示批处理有利，但它不是轨迹加速。c: 每个输出转移的前向映射求值数，quasi-DEER 还另有 JVP；拒绝自环计入转移。d: 进程 CPU 秒/墙钟秒，表示观测消费量而非核数配置。

颈。例如 G1、四链 Picard 把窗口从 16 增至 64 时，首链每轮确认数的组中位数均约 5.57，前向工作却从 5.50 增至 20.88 次/转移，缓存速度比由 0.402 降至 0.170。G2、单链 quasi-DEER 的平均每窗口轮数由 11 增至 22.25，前向工作从 23 增至 45.5 次/转移，速度比由 0.051 降至 0.027。窗口扩大会增加可批处理宽度，但确认推进和求解轮数决定有效工作效率；这些例子是冻结小网格中的观察，尚未识别正加速的交叉点。

为审查阶段组成，另将首链实现拆成单独编译的批量求值/JVP、扫描和接受复核，并记录每轮残差或确认前缀。拆分插入同步及 Python 控制，改变了融合执行。所得控制及未细分部分比例很高，不能据此宣称生产程序主要受 Python 控制限制；它暴露了分段探针的扰动。全部阶段秒数和原始逐轮记录公开，可用于复查数值过程；对生产程序的精确 XLA 剖面仍属于局限。三次独立重复及短微基准不足以给出普遍硬件因果结论。

6.6 可移植复现检查

另建锁定依赖的 Python 环境并安装非 editable 轮子，在迁移目录运行完整测试。首次 57 项通过、1 项因漏带上游比较源文件而失败；补齐许可与该源文件后，仅复查失败项并通过，保留初次失败日志。迁移目录的新目标示例也通过。R 包检查日志与显式 `PB_RUN_INTEGRATION=1` 的 3 项集成测试日志分别交付，后者实际执行 15 个断言，不以默认跳过替代通过。

迁移目录从 1920 项主实验、8 次参考拟合和 320 项 SBC 原始证据重建主摘要，逐字节一致。三个归档含全部源代码、协议、原始数组和论文输入；新临时目录解压后，校验、分析、1928 项现代诊断重算、绘图和编译均通过，诊断值排除耗时后逐项一致。检查仍共享 Mac、R 库和编译器，不等同外部团队或异构系统复现。

7 原生 Windows 扩展：独立开发版本与新冻结实验

开发分支 `windows-native-dev` 新增 Python 版本 0.2.0.dev1 及 R 版本 0.2.0.9001。明确 torch 目标与独立 NumPy 参考可在不导入 JAX 的条件下运行，基础安装不再强制拉取 JAX。本轮实际环境为 Windows 11、AMD Ryzen 7 9800X3D、NVIDIA RTX 5080 (sm_120)、驱动 616.56、Python 3.12.14、torch 2.13.0+cu130 及 float64。R 4.6.1 通过 reticulate 整批调用并保持 `iteration×chain×variable` 次序。Stan 仍为 CPU 提供方式，未自动翻译；本轮未编译核验原生 Stan。Pyro 1.9.2 的 CPU NUTS 仅作为单独正态目标统计基线，未接入 GPU NUTS。

实现保持相同 MALA/RWM 核约定、硬接受前向、独立参考与实际随机数组。quasi-DEER 使用非重叠窗口和截断 Rademacher 对角 JVP；Picard 使用滑动窗口，多链按最小已确认前缀共同推进，末窗口只处理实际转移。控制采用 torch eager 与 Python 循环，记录主机标量等待，不宣称融合设备内控制。CPU 65 项和 CUDA 63 项测试通过，另有 72 项原始路径比较与 8 个 R 批量核验。这些有限测试不构成一般正确性证明。

试探网格 96 项完成后，在观察正式结果前冻结 `windows-native-v1`。协议包含 8 个目标、128/512 两个预算、每组 4 份独立随机数组、CPU/CUDA 资源和 4 个 MH 工作流，共 512 项，每项 4 链、窗口 16。新身份同时锁定源码、依赖、设备、数据、数组、坐标、函数与失败规则。所有 512 项通过数值输出检查，接受事件对独立 NumPy 参考零失配，最大路径差为 5.212×10^{-10} 。256 对 CPU/CUDA 实际数组比较亦无事件失配，最大无约束及约束差分别为 2.842×10^{-14} 及 5.684×10^{-13} 。未发生回退或终结失败。

资源	quasi-DEER/MALA	Online Picard/RWM
CPU	0.101–0.359	0.774–3.560
CUDA	0.164–0.505	1.097–5.230

表 7: 同设备、同 torch、同核的 warmed 执行速度比 (顺序秒数/时间执行器秒数): 各 16 个模型/预算组中位数的最小至最大值。两次技术重放不增加独立重复数。这不是 GPU 相对 CPU 的速度比，也不是精确达标时间。

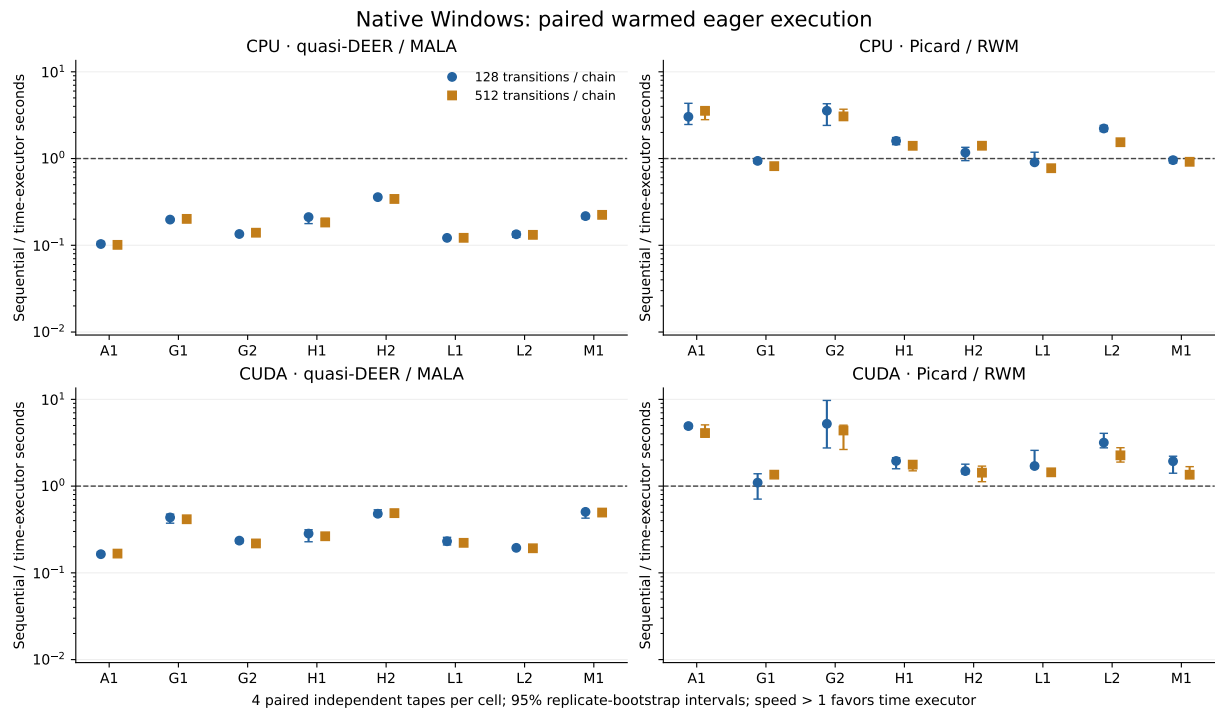


图 6: 原生 Windows 配对执行结果; 每组 4 份独立数组, 误差线为重复层面 2000 次 bootstrap 的逐点 95% 区间, 不提供同时置信保证。CUDA Picard 的 16 组中位数大于 1, 其中 15 组区间下限大于 1。A1/RWM 的全部 32 次拟合均拒绝所有提议, 其速度仅反映自环执行, 不能解释为后验探索加速。quasi-DEER 的所有组中位速度比均小于 1; 全部负结果保留。

数值完成与推断可靠性有明显区别: 480/512 次拟合至少有一个有限 rank/folded Rhat 大于 1.01, 108 次拟合包含常量变量或函数, 其诊断保留为不可判定。最大有限

Rhat 为 4.666，最小有限 bulk/tail ESS 为 4.085/4.205。该 Rhat 计数仅为描述，没有新增事后验收标准。解析目标的逐函数重复 MSE 及区间另行保存；L1/L2 的有限参考未解决，不宣称精度达标。全拒绝或混合不足轨迹没有被删除或重新调参替换。

另以 12 个新生成数据集进行小型 SBC，五个共享数据工作流共 60 次拟合全部完成。同数据解析 90% 区间与五个工作流的覆盖均为 9/12，解析覆盖的 95% 区间为 [0.428, 0.945]；端点、矩和诊断分别保存。共享数据不能计作 60 个独立校准重复。Pyro CPU NUTS 与 MH 不使用固定同轨迹比较，也未纳入本轮正式速度实验。

正式任务记录成本共 1790.220 秒（含普通执行和技术重放），现代 R 诊断另计 28.470 秒；普通 API 包括建模和基本矩；核验 API 使用已构建模型，包含独立参考但不含建模和基本矩。建模、写盘与现代诊断分别记录，这些计时不能混称完整工作流。最大 CUDA 分配/保留峰值为 97,251,328/115,343,360 字节，包含已有驻留分配；CPU 仅有数组空间估算，未测逐拟合 RSS 峰值。主机标量读取累计 88,406 次，标量等待计时 4.206 秒含设备等待，并不隔离全部控制开销。试探分析曾把共享数组的配置变化合并为误差区间；该区间已撤回，原输出及哈希保留，正式分析不受影响。本轮结果仅支持指定 Windows 实现和有限预算下的计算证据，不将 Mac JAX 与 Windows torch 的完整工作流程差异全部归因于 GPU。

回传后的独立复核。 Mac 端校验归档内 5866 个文件，使用冻结 NumPy 参考重放 128 条四链配置并核对全部 512 项实际数组；接受事件仍为零失配，同设备速度比和逐函数误差可由原始记录重建。复算 512 项正式及 60 项 SBC 的现代诊断时发现十进制读取敏感性：NumPy 读取的参数列与保存数组逐值相同，但 Mac R 直接读取 CSV 会改变部分接近重复值的末位精度。以小端 float64 二进制传入 R 后，在绝对及相对 10^{-11} 比较容差下，正式诊断的数值分歧由 4158 个指标值降至 2 个，SBC 由 31 个降至 0 个。剩余 2 个 Rhat 差异约为 3.46×10^{-4} ；后续最小复现定位的数值机制见第 7.1 节。480 项 Rhat 偏高和 108 项常量输出的计数保持不变。这是新增的伴随敏感性分析，保留 Windows 原诊断，不将数值接近的路径等同于秩诊断逐值可复现。该复核没有重新测量 GPU 性能，也没有扩大已验证后端范围。

7.1 收尾阶段新增的诊断与工作量伴随分析

以下分析于 2026 年 10 月 5 日从原始 Windows 记录追加，没有重新采样或更改原冻结协议。二进制输入后剩余的两个 $\hat{R}$ 差异，来自 H2/RWM 顺序、512 步、重复 3 的 CPU 与 CUDA 任务；丢弃 128 步后的 q2 数组逐值相同，故不是两份独立的差异证据。在 Mac R 4.6.0、posterior 1.7.0 中，未折叠分量为 1.1777201954，折叠中位数比精确中点正确舍入值低一个 ULP，约 2.1684×10^{-19} 。固定输入和其余计算，仅替换这个折叠中心，就使 5 个观测的秩改变，折叠 $\hat{R}$ 由 1.4085688369 变为 1.4082223724，后者重现原 Windows 记录。Windows 原生 R 4.6.1/posterior 1.7.0 的实际回执已返回：原生中位数采用正确舍入中点，折叠 $\hat{R}$ 为 1.4082223724，二进制输入和回写相同。这核实了数

值机制及跨系统观测，但未证明某个底层数学库是根因。两种 $\hat{R}$ 都远高于 1.01，不改变混合不足判断，也没有用较小值替换原诊断。12KiB 二进制样例、精确有理数中点、两中心对照和两平台回执随分析源码保存。

对 512 项原记录逐轮核算映射、JVP 和确认前缀，全部计数检查通过。在 16 个模型/预算组的中位数中，quasi-DEER 每输出转移有 11.906–29.500 次前向映射和 5.953–14.750 次 JVP；Picard 有 2.000–6.820 次前向映射，每轮沿每链平均确认 4.493–16.000 步。对应 CPU/CUDA 工作量一致，拒绝自环计入输出转移。映射次数不是密度调用数或 FLOPs，JVP 不能当作一次普通映射定价；这些记录尚不能把总减速因果分解到各个组件。

目标	warmed	速度比	接受率	映射/转移	确认前缀/轮
A1		4.088	0.00000	2.000	16.000
G2		4.380	0.02051	2.293	13.838
G1		1.355	0.49121	6.635	4.763
L1		1.442	0.51611	6.236	5.045

表 8: CUDA Picard、512 步的四个说明性配置；每格 4 份独立数组的中位数。接受率包含丢弃段，分母包括拒绝自环。完整八目标记录另存，未据此筛除任何配置。该表跨目标的几何和求值成本也不同，不能解释为接受率对速度的单独因果效应。

A1 全拒绝时每轮确认整个窗口；G2 的执行加速也伴随很低的接受率。因此执行收益仍须与合理调参后的函数误差共同考察。CUDA 首次执行的标量等待/采样时间比例，quasi-DEER 组中位数范围为 0.970–4.911%，Picard 为 4.476–6.909%；等待含未完成设备工作且没有覆盖全部控制活动，不是独立的主机开销估计。本伴随分析没有混加首次审计与 warmed 重放，也不将两次技术重放算作独立统计重复。上述历史记录不包含窗口/链数干预；独立机制协议的实测结果见第8.1节，共同推断比较仍待完成。

8 跨系统核验与后续推断比较的边界

第二轮接收把无约束路径、参数变换与下游诊断分别核对。108 份 MH 保存路径均在原容差内，接受事件失配为 0；但从同一无约束状态在 Mac 重算原尺度输出时，60 份不满足原逐位相等要求，最大绝对差为 8.88×10^{-15} 。这项严格跨系统检查仍记为失败，不能通过事后添加容差改称全部通过。Windows 同机核验与 Mac 跨系统重算回答不同问题。

CPU Pyro NUTS 在 9 个目标上的串行/四进程技术对照中，54 组保存数组逐字节相同。对收到的 18 份原二进制重新计算 R 诊断，有限 $\hat{R}$ 与 tail ESS 一致，bulk ESS 最大差约 1.02×10^{-12} ，不可判定位置一致。这不意味着重复计算后验函数也必然逐位相同，更不增加独立统计重复数。原短链的探索不足仍然成立。

实际数组传递同样是复现契约的一部分。第二轮启动时，Windows 重新生成的六份候选输入中，噪声与导数探测数组相同，但对数均匀数组有 82/49152 个元素不同，最大差约 4.44×10^{-16} ，因此原协议拒绝启动机制工作流。随后传递原始封存文件，按原哈希和容差完成了下述测量；原启动失败与候选数组仍保留。相同种子和依赖版本不能替代实际数组校验，核验本身也未确定底层数学库差异的因果来源。

8.1 Windows 有限机制测量

独立机制协议在 G1、G2、L1 上各使用 2 份主随机输入，比较窗口 4/16、链数分配及预先规定的 G2 RWM 步长对照。CPU 与 CUDA 各完成 96 个工作流；每个工作流包括首次审计、3 次交错缓存重放和探针后重放。全部 960 份保存路径通过独立 NumPy 参考核验，接受事件零失配，最大绝对路径差为 3.84×10^{-10} 。技术重放、窗口变体和跨设备副本均不增加独立输入数。

测量使用同一台 Windows 11 计算机的 Ryzen 7 9800X3D 和 RTX 5080，torch 主机线程设为 4，CUDA 使用 float64 并显式同步。桌面显示负载仍存在，不将线程数等同独占物理核心数。表9给出全部配置的描述性比值；它们来自三次缓存 sample 时间中位数之比，既不是完整推断成本比，也不是跨独立重复的置信区间。

设备/执行器	配对数	比值大于 1	全部比值范围
CPU / quasi-DEER	24	0	0.092–0.322
CPU / Picard	36	20	0.612–3.574
CUDA / quasi-DEER	24	0	0.139–0.437
CUDA / Picard	36	32	0.778–4.805

表 9: 机制协议的同核缓存执行比值：同组顺序时间除以时间执行器时间。全部 120 个时间执行配置均保留；quasi-DEER 对应 MALA，Picard 对应 RWM。每模型仅两份独立输入，配对数不能解释为独立实验数。

quasi-DEER 每个输出转移对应 7.75–22.5 次前向映射及 3.875–11.25 次 JVP；Picard 为 2–7.59375 次前向映射（图7）。这些计数包括拒绝自环，均高于顺序执行的一次前向映射；映射与 JVP 不是等价浮点工作量，不能简单相加。每端 6 个工作流完全零接受，全部保留。例如 G2/RWM 步长 1.0 的第一份输入，在窗口 4/16 下可一次确认整个窗口，同时没有接受移动。因此较长的确认前缀可以由自环产生，并不表示更有效的后验探索。

预先规定的等 512 个总输出对照提供了另一种解释边界。在 G2、L1 的两份输入及两个设备上，16 条各 32 步的顺序链均比单条 512 步、窗口 16 的 Picard 链更快，后者时间为前者的 4.43–6.92 倍。这 8 个技术对照说明链数分配会改变执行吞吐；两种分配的轨迹长度、初始化份数及探索能力不同，不能解释为相同推断精度下的优劣。图示的

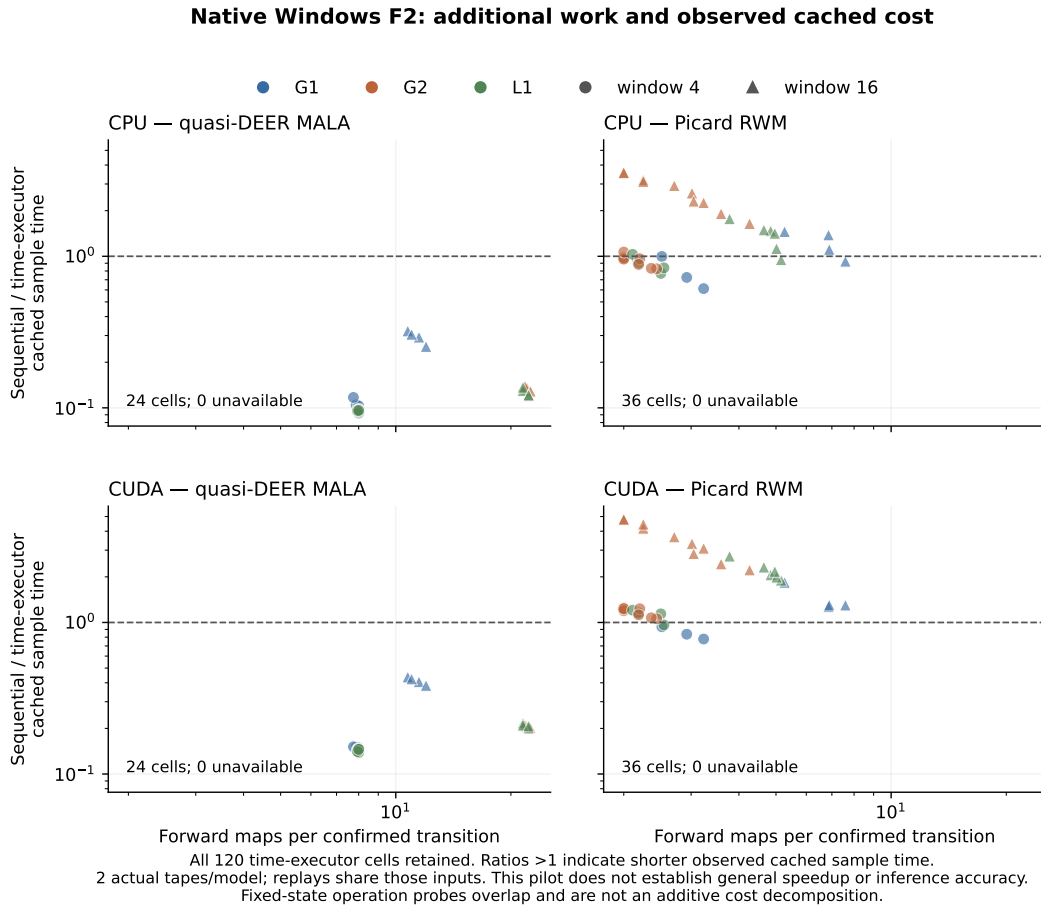


图 7: Windows 机制测量中的重复求值与缓存成本。每点为一个配置与实际输入的组合，共 120 点，部分重叠；颜色区分目标，点形区分窗口。上下行为 CPU/CUDA，左右列为 quasi-DEER/Picard。纵轴大于 1 表示指定缓存执行较短；横轴包括拒绝自环。三次技术重放不扩充独立样本量，图中没有推断精度或统计区间。操作探针存在重叠，不用于构造可加的独占耗时分解。

额外工作量与 quasi-DEER 减速相容，但不足以唯一归因于 JVP、扫描或主机控制；固定状态操作探针与生产路径不同，同步及嵌套操作也使各阶段不能相加还原总时间。

8.2 批次运行与安装的实测范围

另一个 Windows 技术批次完成 27 项主任务和 24 项缓存探测（96 次调用），包含最大形状、进程集合管理和终态恢复检查。接收端对保存 MH 数组独立重放，并核对函数与 R 诊断；这没有新增正式推断重复。90 条函数诊断中仍有 59 条 $\hat{R} > 1.01$ 及 27 条 tail ESS 不可判定，不能将运行成功解释为收敛。该批次支持其封存运行器的技术能力；后续正式协议适配器的独立 Windows 验收与充分预算比较仍需完成。

Python 0.2.0.dev2 与 R 0.2.0.9002 候选已在 Windows 新建依赖环境和仓库外完成安装核验，覆盖 CPU/CUDA MH 和 R 批量入口。默认 R 包检查与显式集成测试分别保存；成功检查不隐去此前安装和 locale 失败。接收端核对源码、sdist、wheel 与 R 包中 17 个模块，并从保存 RDS 重建诊断，没有在 Mac 重新测量 CUDA。该证据不涵盖原生 Windows 的 JAX/Stan 安装、GPU NUTS 或第三方团队的独立复现。

8.3 外部目标与测量接口

水井案例采用 posteriordb 的 3020 个观测，保留原模型中截距和距离系数的平坦先验，而非用带正态先验的内置 logistic 替代。通过两个不同距离上均出现两种结局的观测，可构造可积的似然上界；本数据的后验因而可正规化。Stan、NumPy 和 torch 的密度及导数对照、固定输入轨迹与 R 数组传输已核验，Windows 的 12 项接口工作流也已独立重放。这些是目标和软件扩展证据；短链的明显不良诊断仍保留。有限参考样本与数值求积另行核对，未认证的罕见事件概率不能改写成解析真值。该案例已用于开发，不能再作为未见模型族的泛化检验。

后续充分预算比较需要将可用输出和完整成本对应到同一批重复。为此，已实现三种分别记录的测量口径：准备后、显式同步的内核执行；包含进程启动、输入读取、采样、变换、普通输出与 R 诊断的批量工作流；以及进一步包含独立轨迹参考、资源检查和封存的研究审计调用。后两者为嵌套边界，不能相加。给定核参数和已封存输入的普通工作流不包含事先调参、环境安装或最外层 R 前端；这些费用须另列。后续终态核验也独立登记，不因增加检查次数而改变第一次推断的成本。

一次完整四链拟合是重复单位，链、缓存重放和基础设施重试不增加重复数。任何一次实际尝试缺少计时回执时，该任务的完整成本保持未知，已知部分仍保留。对函数 f ，误差和成本坐标使用同一组数值有效且函数估计有限的重复；其中任何任务缺完整成本，不能通过删去它来生成横坐标。函数不可计算、参考未确定、数值失败和基础设施中断分别记录。共同可用重复上的成对费用比与误差差也采用同一集合，避免混合不同条件样本。

上述归并与统计接口已在保存的任务、故障记录和人工检查样例上验证；它是后续

协议的方法准备，并未追溯替换历史 CPU 或 Windows 首轮的计时与 bootstrap 规则。原生 Windows 的进程集合保护、最大形状和缓存探测已取得上述有限技术批次的实测回执；这些证据不替代后续适配器的独立验收，正式共同精度比较尚未冻结或运行。

9 解释、限制与复现

目前证据支持的是一个能够区分目标一致性、轨迹求解与统计精度的研究实现。Windows CUDA 的 Picard 缓存执行收益与 Mac CPU 负结果对应不同实现、预算和硬件，不能合并成统一速度排名；全拒绝或混合不足也说明，快速重现指定路径可以与较差推断同时发生。历史 Mac CPU 正式比较中，两种时间执行器均未取得组中位 workflow 加速；前向映射调用增加与这一结果一致，说明额外求值和导数计算可能抵消可利用的并行位置。Mac 分段测量与 Windows 机制协议补充了这一解释：Windows quasi-DEER 在全部测量配置中较慢，并记录到每个转移多次前向映射与 JVP；Picard 的部分收益则伴随全拒绝。等总输出对照又表明，增加顺序链数可以比单长链时间执行获得更高吞吐，虽不能据此比较精度。分段同步及重叠探针不能还原生产程序的独占剖面，因此尚未识别 JVP、扫描和主机控制各自的因果贡献。Picard 原文在 Apple M3 CPU 上对昂贵延迟微分方程似然报告约 2.52 倍墙钟加速 [2]；这与本研究的低成本合成目标不同。上游算法的窗口策略、停止规则和计时排除项亦不同，因此当前负例不能解释为 CPU 普遍不适合时间并行，也不识别正加速的分界。

合成模型使目标、真值和失败机制更易核对，同时限制了应用外推。中心化与非中心化漏斗可对照几何，但不能代替真实层次数据分析；潜在 AR(1) 仍具有高斯后验；双峰任务的有限样本模式比例也不证明普遍探索能力。正式结果应据函数、预算和诊断逐项阅读，不能由短链 ESS 或执行器返回成功推断可靠推断已经完成。

quasi-DEER 的局部残差加独立轨迹核验是一种计算上可审查的约束，仍没有提供从容差到一般后验函数偏差的理论界。接受分支接近阈值时，浮点差异可能改变事件；因此本文保存事件差与失败而不以统一 API 隐去保证类别。Online Picard 同样受浮点前缀和与窗口调度影响。本版顺序回退重新计算全路径，尚未优化为仅重算未确认后缀。

历史 Mac 与本轮 Windows 协议评价指定实现、硬件和预算，平台间 workflow 差异不能全部归因于显卡。比较对象也未覆盖现代多链方案：MEADS 和 LAPS 研究群体调参与初始化，有限状态机方法研究多链向量化中的同步开销 [13, 14, 15]。因此，与本研究 NUTS 基线的比较不能决定时间并行相对全部多链策略的优劣。自动配置未进入贡献，水井已参与开发，其正式推断和未见应用泛化仍未验证；当前证据不足以给出通用执行方式选择规则。

代码、数据与辅助工具

历史主实验数据由项目公开生成规则产生；外部水井数据、模型及有限参考来自固定版本 posteriordb[9], 其来源、许可及文件哈希随案例清单保存。源码、测试、协议、分析摘要及论文构建材料已上传至<https://github.com/AlchemistXC/ParallelBayes>；历史 CPU 原始证据见仓库的 `cpu-review-v1` Release。Windows 原始结果归档已上传 `windows-native-v1`, 截至 2026 年 10 月 5 日仍为草稿, 尚不能视为公开可下载材料。接收端已核对归档及内部 5866 个文件。第二轮 Windows 证据已上传另一个草稿并完成独立接收, 本机后续批次与成本伴随归档尚未公开；克隆源码仓库不等于已取得所有原始证据。原始证据、分析源与论文输入以版本和校验和关联；投稿前仍须完成最终公开归档及全部研究结果的统一重建。依赖环境和迁移目录已在 Mac 核验, Windows 候选另取得新环境安装及仓库外运行证据；二者均不等同另一研究团队独立完成全研究复现。R 包检查与显式开启集成测试的日志分开保存。R 包发布候选及 Python 内核随项目交付；上游 BSD 许可源码保留来源, Online Picard 按论文独立实现。本文使用生成式工具辅助代码、实验组织、图表与中文草稿, 并用实际运行、独立实现和可重建原始输出核验相关陈述。稿件尚需研究者最终审阅、确认署名与投稿范围, 不将工具列为作者。

参考文献

- [1] Zoltowski D.M., Wu S., Gonzalez X., et al. [Parallelizing MCMC Across the Sequence Length](#). Advances in Neural Information Processing Systems, 2025, 38.
- [2] Grazi S., Zanella G. [Parallel computations for Metropolis Markov chains with Picard maps](#). Biometrika, 2026, 113(2): asag022.
- [3] Carpenter B., Gelman A., Hoffman M.D., et al. [Stan: A Probabilistic Programming Language](#). Journal of Statistical Software, 2017, 76(1): 1–32.
- [4] Roualdes E.A., Ward B., Carpenter B., Seyboldt A., Axen S.D. [BridgeStan: Efficient in-memory access to the methods of a Stan model](#). Journal of Open Source Software, 2023, 8(87):5236. 本研究使用版本 2.7.0, 接口说明及构建来源另行归档。
- [5] Cabezas A., Corenflos A., Lao J., Louf R., et al. [BlackJAX: Composable Bayesian inference in JAX](#). arXiv:2402.10797, 2024.
- [6] Bingham E., Chen J.P., Jankowiak M., et al. [Pyro: Deep Universal Probabilistic Programming](#). Journal of Machine Learning Research, 2019, 20(28):1–6.
- [7] NIMBLE development team. [compareMCMCs: Tools for comparing MCMC efficiency from NIMBLE and other engines](#). 软件作者仓库, 访问于 2026 年 10 月 3 日；此处引用功能说明, 未复现其性能。

- [8] Kulkarni S., Shah K.D., Arora N., et al. [PPL Bench: Evaluation Framework For Probabilistic Programming Languages](#). arXiv:2010.08886v1, 2020. 软件来源: <https://github.com/facebookresearch/pplbench>。
- [9] Magnusson M., Torgander J., Bürkner P.-C., Zhang L., Carpenter B., Vehtari A. [posteriordb: Testing, Benchmarking and Developing Bayesian Inference Algorithms](#). Proceedings of Machine Learning Research, 2025, 258:1198–1206.
- [10] Stan development team. [posterior: Rhat, bulk ESS and tail ESS documentation](#). 本研究使用 posterior 1.7.0; 常量链诊断保留为不可判定。
- [11] Senne R. and contributors. [ParallelMCMC.jl](#). 软件仓库, 提交 7be9c86, 访问于 2026 年 10 月 6 日; 引用接口功能, 未运行性能比较。
- [12] Sosa S. [BayesForge: Bayesian Inference using numpyro and XLA](#). R 包 0.0.1 参考手册, 访问于 2026 年 10 月 6 日; 引用建模及后端接口。
- [13] Hoffman M.D., Sountsov P. [Tuning-Free Generalized Hamiltonian Monte Carlo](#). Proceedings of Machine Learning Research, 2022, 151:7799–7813.
- [14] Robnik J., Seljak U. [Faster Parallel MCMC: Metropolis Adjustment Is Best Served Warm](#). Proceedings of Machine Learning Research, 2026, 300:2719–2727.
- [15] Dance H., Glaser P., Orbanz P., Adams R.P. [Efficiently Vectorized MCMC on Modern Accelerators](#). Proceedings of Machine Learning Research, 2025, 267:12436–12458.